

1. What is the purpose of the JVM, and how does it differ from the JRE?

Answer:

The JVM (Java Virtual Machine) executes Java bytecode, while the JRE (Java Runtime Environment) provides the necessary libraries and resources for running Java applications.

2. Describe the steps to set up a Java development environment on your system.

Answer:

You need to install the JDK (Java Development Kit), configure the PATH environment variable, and set up an IDE like IntelliJ or Eclipse.

3. What is the command to compile and run a Java program from the command line?

Answer:

Use `javac filename.java` to compile and `java classname` to run the program.

```
javac HelloWorld.java  
java HelloWorld
```

4. Explain the role of the main method in a Java program.

Answer:

The main method is the entry point of a Java application where execution begins.

```
public static void main(String[] args) {  
    System.out.println("Hello, World!");  
}
```

5. What is the difference between javac and java commands?

Answer:

`javac` compiles the Java source code into bytecode, while `java` runs the compiled bytecode on the JVM.

6. How do you declare a constant in Java?

Answer:

Use the `final` keyword to declare a constant whose value cannot change after initialization.

```
final int MAX_SPEED = 120;
```

7. What is type casting, and how is it done in Java?

Answer:

Type casting is converting one data type to another. Explicit casting is required when narrowing from a larger to a smaller type.

```
int i = (int) 10.5; // Explicit casting from double to int
```

8. Explain the difference between String, StringBuilder, and StringBuffer.

Answer:

String is immutable, StringBuilder is mutable and not thread-safe, while StringBuffer is mutable and thread-safe.

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" World");
```

9. How can you convert a char to an int in Java?

Answer:

You can directly assign a char to an int, as char is internally represented as an integer (ASCII value).

```
char c = 'A';  
int i = c; // i = 65
```

10. What will happen if you try to assign a double value to an int variable without casting?

Answer:

The code will result in a compilation error, as Java requires explicit casting when converting from double to int.

11. Write a Java program to check if a number is even or odd using the modulo operator.

Answer:

The program uses the modulo operator % to determine if a number is divisible by 2.

```
int num = 10;  
if (num % 2 == 0) {  
    System.out.println("Even");  
} else {  
    System.out.println("Odd");  
}
```

12. How do you implement a switch-case statement to handle multiple conditions in Java?

Answer:

Switch-case allows multiple possible values for a single variable, each handled by a different case.

```
int day = 3;
switch (day) {
case 1: System.out.println("Monday"); break;
case 2: System.out.println("Tuesday"); break;
case 3: System.out.println("Wednesday"); break;
default: System.out.println("Invalid day");
}
```

13. Explain the difference between while and do-while loops in Java.

Answer:

while checks the condition before executing the loop body, whereas do-while checks the condition after executing the loop body.

```
int i = 0;
do {
System.out.println(i);
i++;
} while (i < 5);
```

14. Write a Java program to find the largest of three numbers using if-else statements.

Answer:

The program compares the three numbers using nested if-else conditions.

```
int a = 10, b = 20, c = 15;
if (a >= b && a >= c) {
System.out.println("Largest: " + a);
} else if (b >= a && b >= c) {
System.out.println("Largest: " + b);
} else {
System.out.println("Largest: " + c);
}
```

15. How can you exit a loop prematurely in Java?

Answer:

Use the break statement to exit a loop prematurely based on a certain condition.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break;  
    }  
    System.out.println(i);  
}
```

16. Write a Java program to reverse an array of integers.

Answer:

The program swaps elements from the start and end of the array until the middle is reached.

```
int[] arr = {1, 2, 3, 4, 5};  
for (int i = 0; i < arr.length / 2; i++) {  
    int temp = arr[i];  
    arr[i] = arr[arr.length - 1 - i];  
    arr[arr.length - 1 - i] = temp;  
}
```

17. How do you sort an array of strings in Java?

Answer:

You can use `Arrays.sort()` to sort an array of strings in lexicographical order.

```
String[] arr = {"Banana", "Apple", "Cherry"};  
Arrays.sort(arr);
```

18. Write a Java program to find the frequency of a character in a string.

Answer:

The program iterates through the string and counts occurrences of the specified character.

```
String str = "hello world";  
char ch = 'o';  
int count = 0;  
for (int i = 0; i < str.length(); i++) {  
    if (str.charAt(i) == ch) {  
        count++;  
    }  
}  
System.out.println("Frequency: " + count);
```

19. How do you concatenate two strings in Java?

Answer:

You can use the + operator or the concat() method to concatenate strings.

```
String s1 = "Hello";  
String s2 = "World";  
String s3 = s1 + " " + s2;
```

20. Write a Java program to check if two strings are anagrams.

Answer:

The program sorts both strings and compares them for equality.

```
String s1 = "listen";  
String s2 = "silent";  
char[] arr1 = s1.toCharArray();  
char[] arr2 = s2.toCharArray();  
Arrays.sort(arr1);  
Arrays.sort(arr2);  
if (Arrays.equals(arr1, arr2)) {  
    System.out.println("Anagrams");  
} else {  
    System.out.println("Not Anagrams");  
}
```

21. What is encapsulation in Java, and how do you implement it?

Answer:

Encapsulation is the bundling of data and methods within a class, and restricting access using access modifiers.

```
public class Person {  
    private String name;  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
}
```

22. Explain the difference between method overloading and method overriding.

Answer:

Method overloading occurs when multiple methods have the same name but different parameters, while overriding occurs when a subclass redefines a method from its superclass.

```
// Overloading  
public int add(int a, int b) { return a + b; }  
public int add(int a, int b, int c) { return a + b + c; }
```

```
// Overriding
@Override
public String toString() { return "Custom String"; }
```

23. Write a Java program to demonstrate single inheritance.

Answer:

Single inheritance involves a child class inheriting from a single parent class.

```
class Animal {
void sound() { System.out.println("Animal Sound"); }
}
class Dog extends Animal {
void bark() { System.out.println("Dog Barks"); }
}
Dog dog = new Dog();
dog.sound();
dog.bark();
```

24. What is an abstract class in Java, and how do you use it?

Answer:

An abstract class is a class that cannot be instantiated and may contain abstract methods that subclasses must implement.

```
abstract class Shape {
abstract void draw();
}
class Circle extends Shape {
void draw() { System.out.println("Drawing Circle"); }
}
```

25. Explain the concept of polymorphism with an example.

Answer:

Polymorphism allows objects to be treated as instances of their parent class, enabling dynamic method invocation.

```
class Animal {
void sound() { System.out.println("Animal Sound"); }
}
class Dog extends Animal {
void sound() { System.out.println("Dog Barks"); }
}
```

```
Animal animal = new Dog();  
animal.sound(); // Output: Dog Barks
```

26. Write a Java program to handle division by zero using exception handling.

Answer:

The program uses a try-catch block to catch the `ArithmeticException`.

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

27. What is the difference between checked and unchecked exceptions?

Answer:

Checked exceptions are checked at compile-time, while unchecked exceptions are checked at runtime.

```
// Checked Exception  
try {  
    FileReader file = new FileReader("test.txt");  
} catch (FileNotFoundException e) {  
    e.printStackTrace();  
}
```

```
// Unchecked Exception  
int num = 10 / 0;
```

28. How do you create a custom exception in Java?

Answer:

You create a custom exception by extending the `Exception` class.

```
class MyException extends Exception {  
    MyException(String s) { super(s); }  
}
```

29. Explain the use of the finally block in exception handling.

Answer:

The finally block contains code that executes regardless of whether an exception occurs or not.

```
try {
int data = 10 / 0;
} catch (ArithmeticException e) {
System.out.println(e);
} finally {
System.out.println("Finally block executed");
}
```

30. What happens if an exception is not handled in a Java program?

Answer:

If an exception is not handled, the program will terminate abnormally, and the JVM will display a stack trace.

31. Write a Java program to add elements to an ArrayList and print them.

Answer:

The program creates an ArrayList, adds elements to it, and iterates over them using a for loop.

```
ArrayList<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
list.add("Cherry");
for (String fruit : list) {
System.out.println(fruit);
}
```

32. Explain the difference between HashMap and TreeMap in Java.

Answer:

HashMap stores elements in an unordered way, while TreeMap stores elements in a sorted order based on the natural ordering of keys.

```
Map<String, Integer> map = new HashMap<>();
map.put("Apple", 1);
map.put("Banana", 2);
```

```
Map<String, Integer> treeMap = new TreeMap<>(map);
```

33. Write a Java program to demonstrate the use of a HashSet.

Answer:

The program adds elements to a HashSet and iterates over them using an iterator.

```
HashSet<String> set = new HashSet<>();
set.add("One");
set.add("Two");
set.add("Three");
Iterator<String> it = set.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

34. How do you use a PriorityQueue in Java?

Answer:

PriorityQueue orders elements according to their natural ordering or a custom comparator.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(10);
pq.add(20);
pq.add(15);
System.out.println(pq.poll()); // Output: 10
```

35. What is the difference between Iterator and ListIterator?

Answer:

Iterator can traverse in forward direction only, while ListIterator can traverse both forward and backward.

```
ListIterator<String> listIterator = list.listIterator();
while (listIterator.hasNext()) {
    System.out.println(listIterator.next());
}
```

36. Write a Java program to read a file line by line using BufferedReader.

Answer:

The program uses BufferedReader to read each line of a file until the end.

```
BufferedReader reader = new BufferedReader(new FileReader("test.txt"));
String line;
while ((line = reader.readLine()) != null) {
    System.out.println(line);
}
reader.close();
```

37. How do you write text to a file using FileWriter in Java?

Answer:

The program uses `FileWriter` to write a string to a text file.

```
FileWriter writer = new FileWriter("output.txt");  
writer.write("Hello, World!");  
writer.close();
```

38. Explain the concept of serialization in Java.

Answer:

Serialization is the process of converting an object into a byte stream to save it to a file or send it over a network.

```
FileOutputStream fileOut = new FileOutputStream("data.ser");  
ObjectOutputStream out = new ObjectOutputStream(fileOut);  
out.writeObject(object);  
out.close();
```

39. Write a Java program to append text to an existing file.

Answer:

The program uses `FileWriter` in append mode to add text to the end of a file.

```
FileWriter writer = new FileWriter("output.txt", true);  
writer.write("Appending text");  
writer.close();
```

40. What is the purpose of the `File` class in Java?

Answer:

The `File` class represents file and directory pathnames in an abstract manner and provides methods to manipulate them.

```
File file = new File("test.txt");  
if (file.exists()) {  
    System.out.println("File exists");  
}
```

41. Write a Java program to create a thread by extending the `Thread` class.

Answer:

The program creates a thread by extending the `Thread` class and overriding the `run()` method.

```

class MyThread extends Thread {
public void run() {
System.out.println("Thread is running");
}
}
MyThread t1 = new MyThread();
t1.start();

```

42. How do you create a thread by implementing the Runnable interface?

Answer:

The program creates a thread by implementing the Runnable interface and passing it to a Thread object.

```

class MyRunnable implements Runnable {
public void run() {
System.out.println("Thread is running");
}
}
Thread t1 = new Thread(new MyRunnable());
t1.start();

```

43. What is thread synchronization, and how is it achieved in Java?

Answer:

Thread synchronization ensures that only one thread can access a resource at a time, typically achieved using the synchronized keyword.

```

class Counter {
synchronized void increment() {
count++;
}
}

```

44. Write a Java program to demonstrate inter-thread communication using wait() and notify().

Answer:

The program uses wait() to pause a thread and notify() to wake it up when a condition is met.

```

class Chat {
synchronized void question() throws InterruptedException {
wait();
System.out.println("Answered");
}
synchronized void answer() {

```

```
notify();  
}  
}
```

45. Explain the lifecycle of a thread in Java.

Answer:

A thread goes through New, Runnable, Running, Blocked, and Terminated states during its lifecycle.

```
Thread t = new Thread();  
t.start(); // Runnable to Running  
t.sleep(1000); // Running to Blocked
```

46. How do you handle concurrency issues in Java?

Answer:

Concurrency issues can be handled using synchronization, volatile variables, and high-level concurrency utilities like `java.util.concurrent` package.

```
// Using synchronized block  
synchronized (this) {  
    sharedResource++;  
}
```

47. What is the difference between `start()` and `run()` methods in Java threads?

Answer:

`start()` creates a new thread and calls `run()` on that thread, whereas calling `run()` directly executes the method in the current thread.

```
// Using start()  
Thread t = new Thread(() -> System.out.println("Thread running"));  
t.start();
```

48. Write a Java program to implement a simple thread-safe counter.

Answer:

The program uses the `synchronized` keyword to make increment operations on a shared counter thread-safe.

```
class Counter {  
    private int count = 0;  
    public synchronized void increment() {  
        count++;  
    }  
}
```

```
}  
public int getCount() {  
    return count;  
}  
}
```

49. How do you create a thread pool in Java?

Answer:

Use the Executors class to create a fixed-size thread pool for executing tasks.

```
ExecutorService executor = Executors.newFixedThreadPool(5);  
executor.execute(() -> System.out.println("Task executed"));  
executor.shutdown();
```

50. What is the purpose of the volatile keyword in Java?

Answer:

The volatile keyword ensures visibility of changes to variables across threads, preventing them from being cached.

```
private volatile boolean flag = true;
```